

LangGraph & MAF Lab

TWO FRAMEWORKS · SAME PROBLEM · HANDS-ON COMPARISON

LAB 01 · BUILD A MULTI-AGENT TICKET ROUTER

01 SCAFFOLD

```
% pip install langgraph agent-framework
% mkdir lab-tickets && cd lab-tickets
% git init && touch .env
```

02 CONTRACT

```
class Ticket(TypedDict):
    id: str; priority: int; topic: str
    history: List[Message]; assignee: str
```

03 RUN + TRACE

```
app.invoke(input, config={"thread_id": ...})
for evt in app.stream(input): trace.append(evt)
```

CHOOSE A FRAMEWORK

LangGraph

graph · cycles · checkpointer
time-travel · open-source
state reducers · DIY ops
→ focuses on control logic

MS Agent Framework

group-chat · handoffs · Magentic
managed ledger · Entra ID
OpenTelemetry hooks native
→ focuses on governance

04 VERIFY

```
pytest tests/test_router.py -k trace
ai studio eval → faithfulness ≥ 0.9
```

05 DEPLOY

```
az containerapp up --source .
azd pipeline → B5 release
```

06 OBSERVE

```
app insights · otel trace
dashboard: latency, $cost, accuracy
```

LEARNING GOALS

- Translate a process into a typed state schema
- Pick LangGraph or MAF by control needs
- Wire a MemorySaver / Foundry checkpoint
- Inspect a stream of node events

DELIVERABLES

- Runnable notebook + service code
- Trace diagram (PNG + JSON events)
- Eval set v1 (10 routing cases)
- README w/ choose-framework table

StateGraph Hands-On

NODES · REDUCERS · CONDITIONAL EDGES · CHECKPOINTER

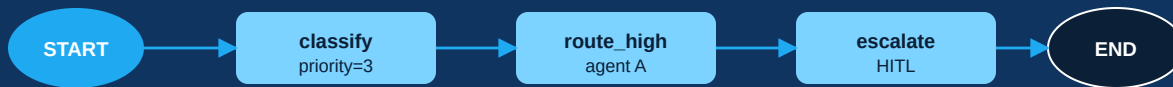
lab-tickets · graph.py

```
class TicketState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]
    priority: int; assignee: str
    trace: Annotated[List[dict], operator.add]

def classify(state: TicketState) -> TicketState:
    label = llm.predict(state["messages"])
    return {"trace": [{ "node": "classify", "label": label }]}

graph = StateGraph(TicketState)
graph.add_node("classify", classify)
graph.add_node("route_low", route_low)
graph.add_node("route_high", route_high)
graph.add_node("escalate", escalate)
graph.add_conditional_edges("classify", by_priority,
    {3: "route_high", 2: "route_low", 1: "escalate"})
app = graph.compile(checkpointer=MemorySaver())
```

EXECUTION TRACE (replay from checkpoint)



```
>>> app.invoke({"messages": [...]}, config={"thread_id": "T-42"})
→ classify   t=0.3s  priority=3
→ route_high t=0.5s  agent=A
→ escalate  t=0.2s  user=ops@
>>> app.get_state(config).values["trace"]
[{"node": "classify"}, {"node": "route_high"}, {"node": "escalate"}]
```

HANDS-ON TASKS

1. Type `TicketState`
2. Implement `classify` with prompt
3. Add `conditional_edge`
4. Re-run, inspect trace JSON

PITFALLS TO WATCH

- **Reducer drift:** forgetting `operator.add`
- Conditional key not in `NODE_MAP`
- `State.update()` returns dict, not partial
- Checkpoint id missing → no replay

MS Agent Framework Workflows

GROUP-CHAT · HANDOFF · CONCURRENT · MANAGED LEDGER

lab-tickets · workflow.py

```
from agent_framework import ChatAgent, GroupChatOrchestrator, handoff

triage = ChatAgent(name="triage", instructions="route by topic")
billing = ChatAgent(name="billing", tools=[refund_tool])
tech = ChatAgent(name="tech", tools=[k8s_reboot, runbook_search])
reviewer = ChatAgent(name="reviewer", instructions="QA every answer")

wf = GroupChatOrchestrator(
    participants=[triage, billing, tech, reviewer],
    speaker_policy="dynamic",
    max_rounds=6,
    exit_on=handoff(reviewer),
)

async for evt in wf.run_stream(thread_id="T-42", input=message):
    ledger.append(evt) # 0tel-exported
    print(evt.source, ">", event.payload["text"])
```

RUN-STREAM EVENTS · WITHIN ONE THREAD



```
ledger[thread=T-42]: ack triage(0.2s) · ack tech(0.8s) · ack reviewer(0.4s) · sig reviewer(0.1s)
```

LAB TASK PLAN

1. Define 4 agents + tools
2. Wire `GroupChatOrchestrator`
3. Add `handoff()` exit gate
4. Stream events, dump ledger
5. Replay via `thread_id`

TRY VARIANTS

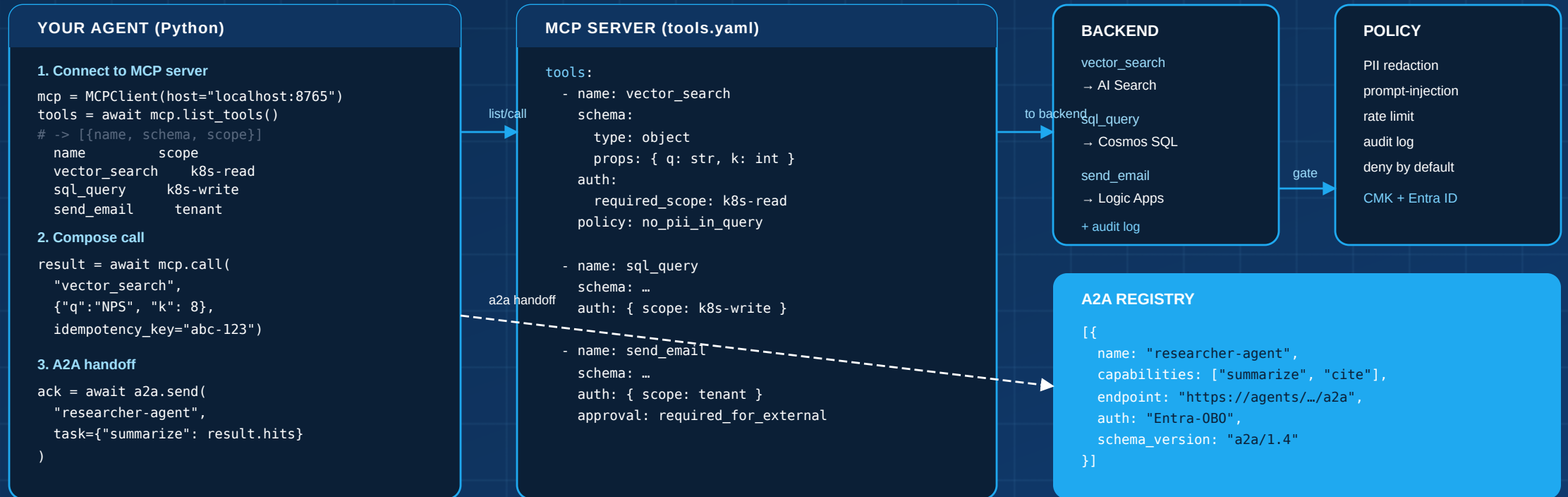
- Concurrent fan-out (parallel)
- Magentic mode (plan + ledger)
- Approval gate at reviewer

COMPARE

Same problem set as LangGraph — note governance overhead vs. control freedoms.

MCP & A2A Integration Lab

BUILD A TOOL SERVER · DISCOVER · CALL · CHAIN



LAB OBJECTIVES

Build MCP server · Connect agent · Tool call w/ scoped auth · A2A handoff

EXIT CRITERIA

Policy rejects unknown scope · Idempotency succeeds on retry · Trace visible in ledger

Tool Server · Discovery · Versioning

PROTOCOL UPGRADES · NAMESPACE · AGENT CARDS

DISCOVERY · NAMESPACE · VERSION FLOW

01 / LIST

agents.ask(
 capability=
 "search")

REGISTRY RESPONSE

```
200 OK [agentCard] schema_version: a2a/1.4 capabilities: ["search","summarize","cite"]  
          endpoint: https://agents/api/.../a2a auth: Entra-OB0 sla: {p95: 1.2s}
```

02 / NAMESPACE

tools = by
 tenant_id
 resource_scope

NAMESPACE MERGE

```
tenant.eu.legal.k8s.read → tool defs: {sql_query, doc_search} tenant.eu.legal.write → ...  
                         (tool schema) u (auth scope) u (rate limit) = agent-runtime capabilities
```

03 / VERSION

schema_v
 compat
 deprecate

VERSION TABLE

```
a2a/1.4 : current supported through 2027-Q2  
a2a/1.3 : maintenance window  
a2a/1.2 : deprecated — fail-fast on connect
```

04 / WIRE

HAPPY PATH CALL

```
result = await mcp.call("sql_query", {"q":"..."}, scope="tenant.eu", auth="entra-obo", idem="r-7")
```

DISCOVERY COMPARISON

ASPECT	MCP	A2A
Unit	tool	agent
Card	tool schema	agentCard
Auth	scope + idem	Entra-OBO
Version	schema_v	schema_version

HANDS-ON TIPS

- Version **before** you merge a tool change
- Use a [registry](#) not ad-hoc URLs
- Always carry [idempotency_key](#)
- Idempotent retries on 5xx — not on 4xx
- Reject on missing [scope](#)

Voice Agent Lab

STREAMING · VAD · Barge-in · TURN TAKING · GUARDRAILS



BUILD

Wire STT → Agent → TTS · Add VAD · Set persona

EXERCISE

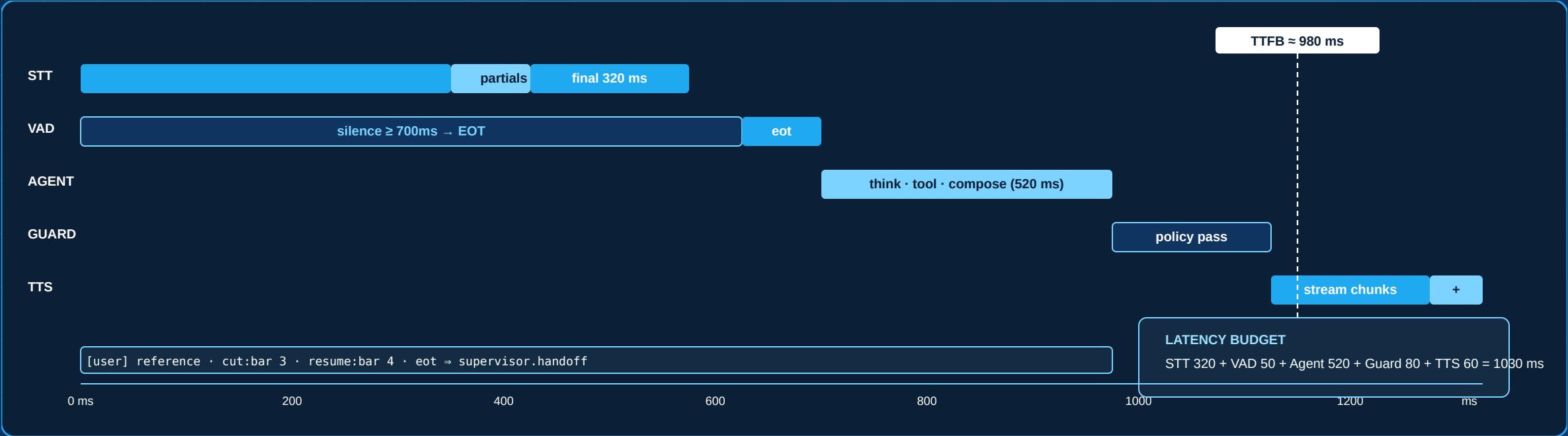
10 scripted turns · 1 barge-in · 1 escalation

PASS CRITERIA

Barge-in ≤ 600 ms · EOT acc ≥ 0.85 · CSAT ≥ 4.0

Voice Pipeline: STT · VAD · Agent · TTS

STREAMING CONTRACTS · BUFFER DEPTHS · INTERRUPTIONS



BUFFERS

- STT ring: 800 ms (200 ms hop)
- VAD window: 300 ms
- Agent prefetch: idem key > token
- TTS queue: 2 sentences ahead

TURN RULES

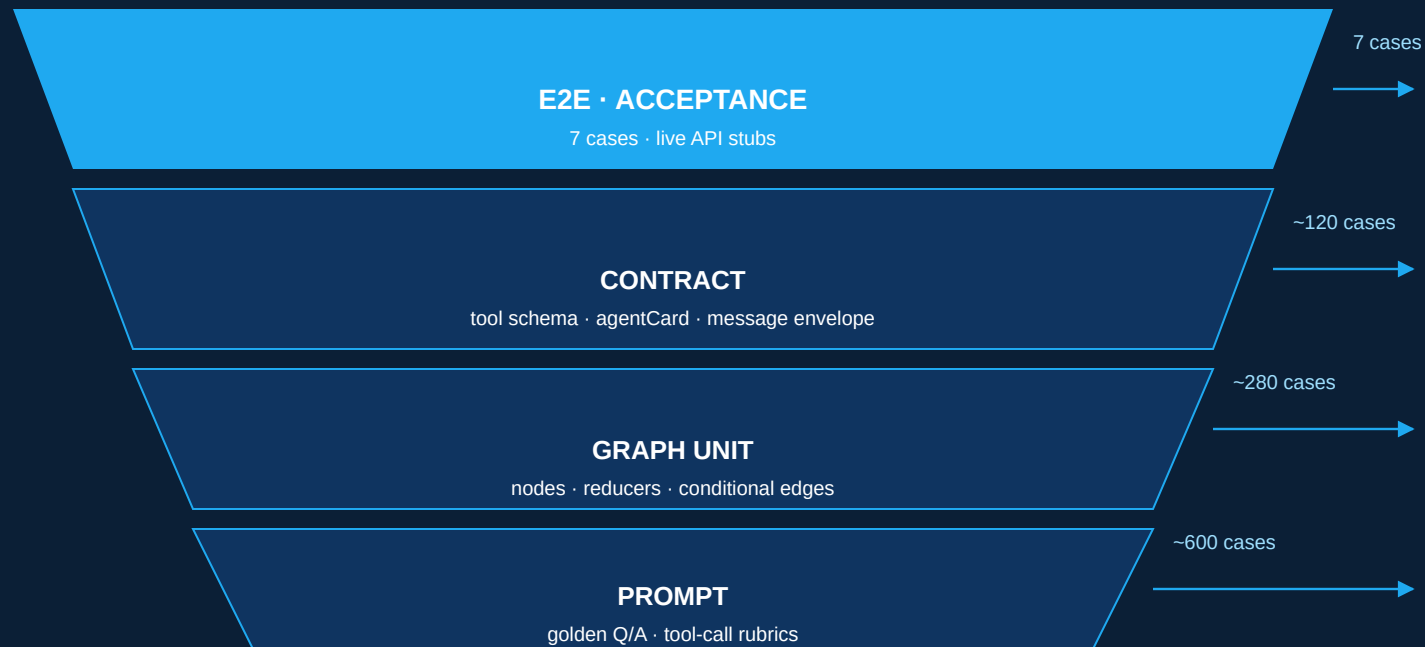
- EOT trigger dual: silence + LM hint
- Barge-in: stop TTS, replay last 1.2 s
- Loops: drop into human after 3 turns
- Pause budget per turn $\leq 1.2\text{ s}$

FAILURE MODES

- Echo $\rightarrow$ caller hears their own TTS
- False EOT $\rightarrow$ mid-word cut
- Agent sprawl $\rightarrow$ long tokens
- TTS clipping $\rightarrow$ flushed phrase

Testing Agents

UNIT · CONTRACT · END-TO-END · TRACE REPLAY

**RUNNER (pytest)**

```
pytest -q tests/contract tests/unit | pytest --trace=spans tests/e2e | diff ./trace/golden.json ./trace/curr.json
```

PER-LAYER ASSERTIONS

- **Reducer:** commutative, idempotent
- **Node:** input → output invariants
- **Edge:** valid keys, deterministic cycle
- **Tool:** schema + ACL + idempotency
- **Run:** ≤N steps, ≤\$cost cap, ≤latency

LAB EXERCISE

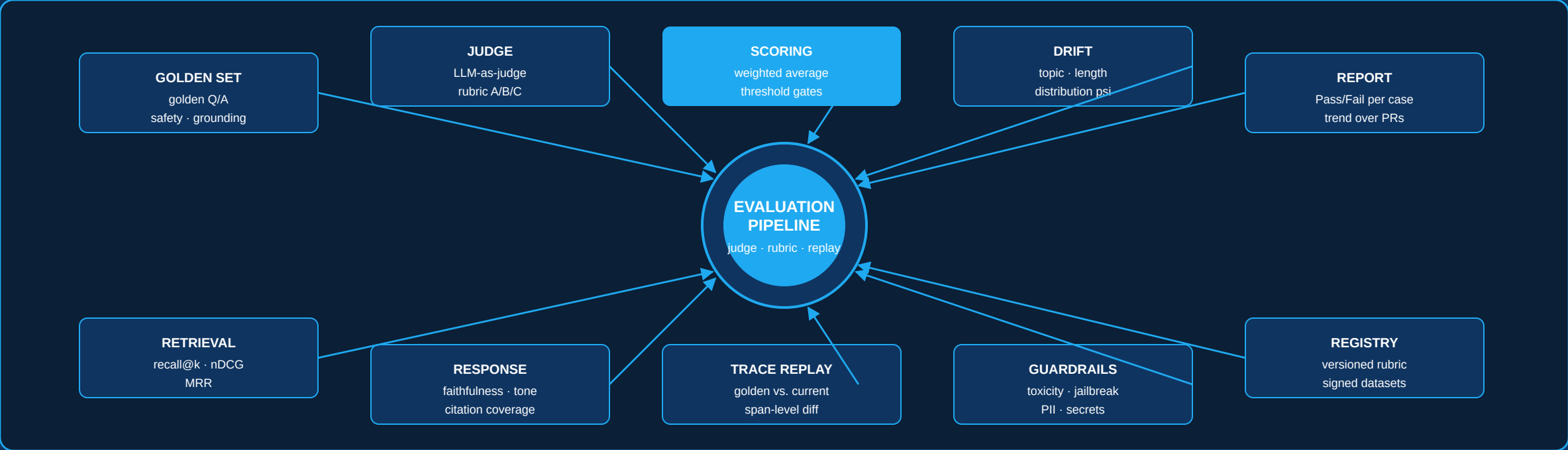
1. Write reducer unit test (50 L)
2. Add contract test for 3 tools
3. Snap a trace, mutate prompt, replay
4. Patch diff → fail/pass verdict

PASS CRITERIA

Trace golden diff == empty · 100% tools covered · all guards active

Evaluation Framework

METRICS BY LAYER · JUDGE MODELS · GOLDEN SETS



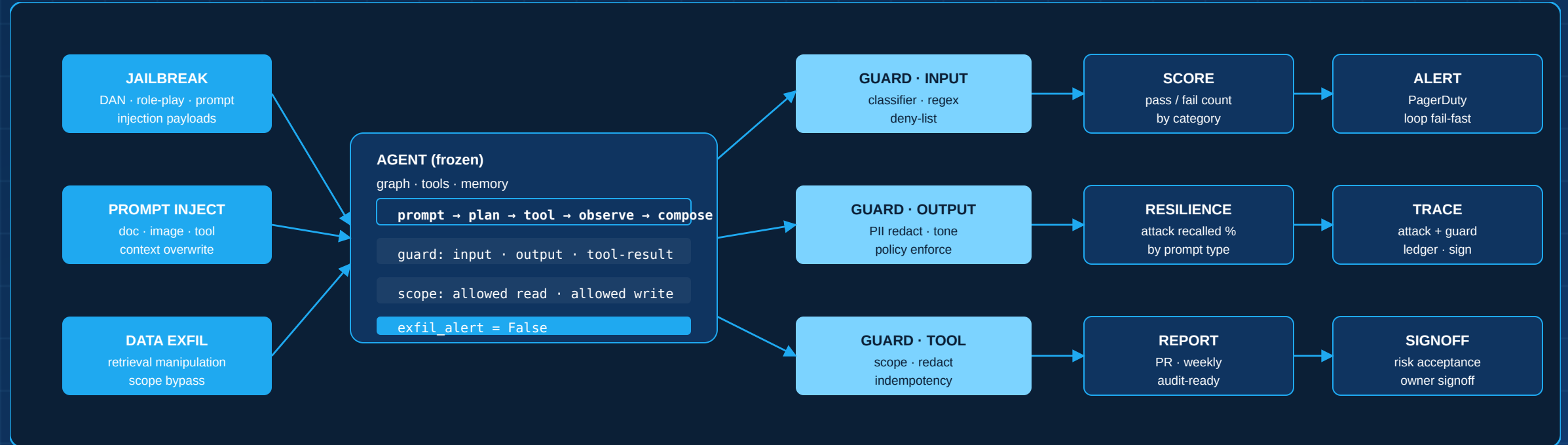
JUDGE PIPELINE (CI GATE)

git push → eval_harness pulls golden@ · run agent v_N+1 with frozen model · judge scores against rubric@v · gate must pass · traces archived · report PR comment.

Hard gates: faithfulness ≥ 0.92 · jailbreak < 0.5% · PII leak = 0 · \$cost ≤ budget · steps ≤ 12.

Red-Team & Safety Suites

ADVERSARIAL PROMPTS · GUARDRAIL TRIPS · STRESS RUNS



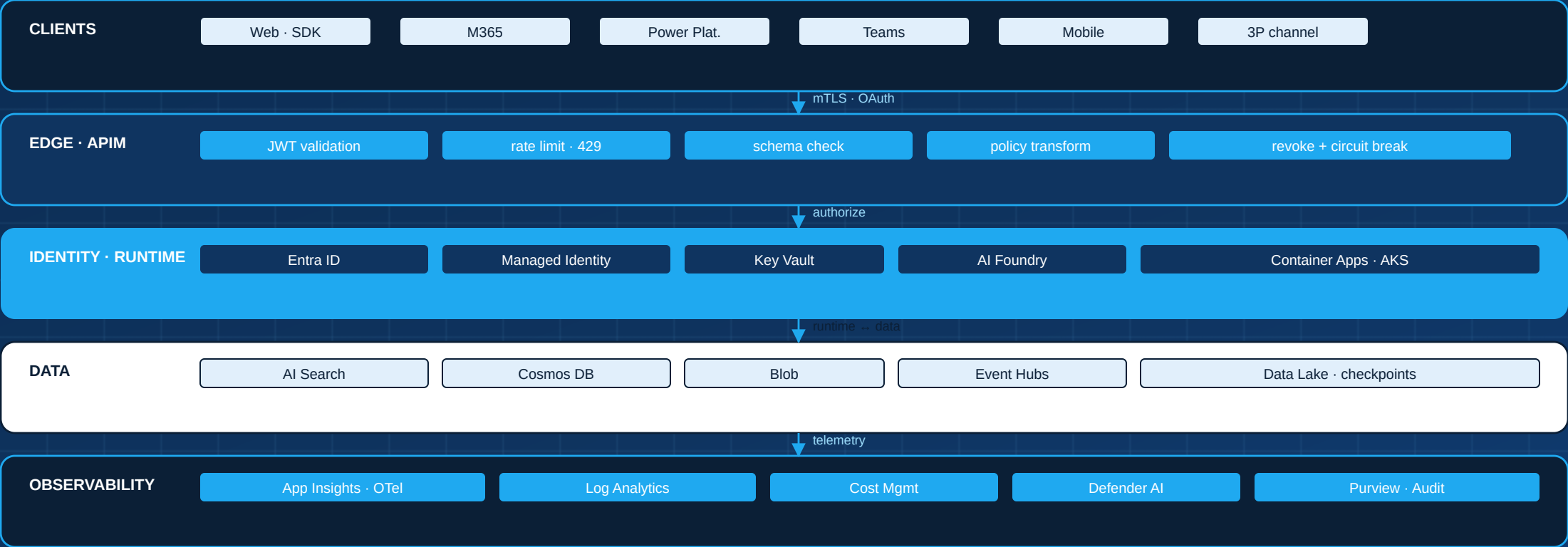
LAB WALKTHROUGH

1. Run `redteam.py` against your agent · 2. Inspect ledger for guardrail trips · 3. Patch a guard · 4. Re-run, observe >0% recall lift.

Pass criteria: 0 jailbreak, 0 PII leak, 0 exfil, ≤1% scope drought, every guard trip archived.

Deploying Agents

CONTAINERS · APIM · IDENTITY · REGIONS · COST



AZD TEMPLATE

azd up · Container Apps · APIM · AI Search · Cosmos

REGIONS

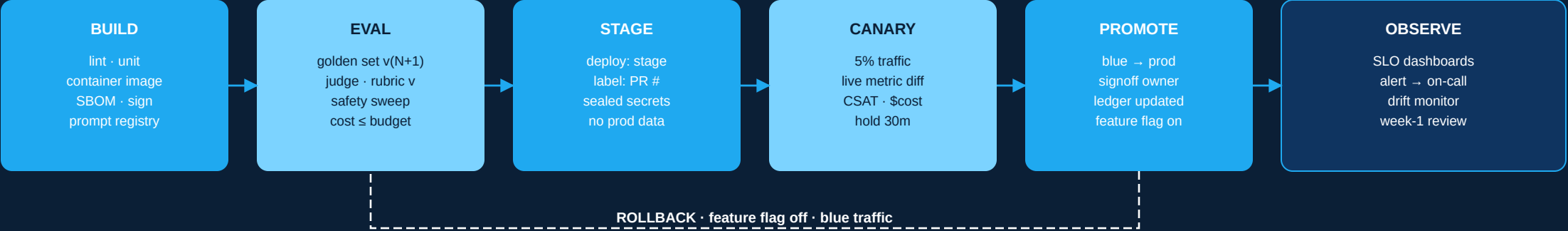
eastus2 (PTU) · westeurope (batch) · multi-region DR

COST LEVERS

PTU reservation · batching · token quota · idle scale-to-zero

Release Pipeline

CI · EVAL · CANARY · BLUE-GREEN · ROLLBACK



PROMOTION GUARDS



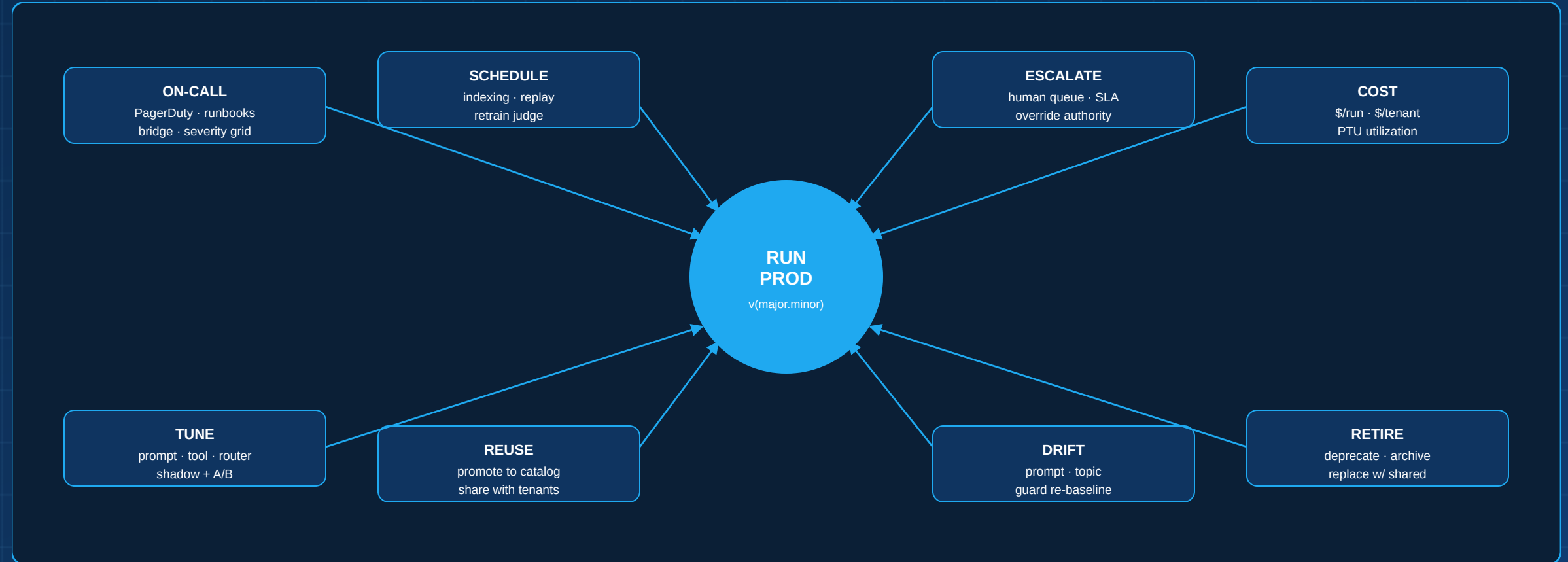
each guard is a CI step; failure blocks promote. Signoffs in PR & audit log.
if guards breach in canary → auto-rollback, fire alert, attach trace.

HANDS-ON

PR → CI runs · fix 1 failing guard → push · observe green promote → invalidate one metric to demo rollback

Operating Agents · Day 2+

ON-CALL · TUNING · RETIRE · RE-USE · COST



SLOS

faith ≥ 0.92 · CSAT ≥ 4.0 · p95 $\leq 1.2s$ · \$/run $\leq \$0.40$ · guard-trip $\leq 1\%$

RUNBOOK HEADLINES

feature-flag off → drain → freeze model → rollback checkpoint → re-baseline

Observability Deep Dive

OTEL · SPANS · LOGS · METRICS · COST METER

TRACES

span graph

node timeline

tool-call spans

latency per step

OTEL · W3C trace context
thread_id · user · tenant
tool schema version
prompt hash · model id

METRICS

TTFB p50 / p95

tokens/run · \$/run

guard-trip %

tool error rate

counters · gauges · histograms
per-tenant · per-prompt
SLO burn rate
PTU utilization

LOGS

tool result · 1-line

guard verdict

policy hit

approval · handoff

structured · JSON · signed
PII redacted by default
chain-of-custody id
retention · per region

SRE DASHBOARD

TTFB p50	520 ms	▲-3.2%
TTFB p95	1.18 s	◇
\$/run	\$0.34	◇
faith	0.93	▲+0.4%
CSAT	4.4/5	▲+0.1
guard-trip	0.7%	◇
tool-fail	0.4%	◆down
PII leak	0	◆
burn (faith)	0.18	◇
open alerts	0	◇

→ App Insights

→ Log Analytics

→ Grafana

SLI / SLO

SLI = measured signal · SLO = commitment · Error Budget = 1 - SLO · Burn = pace vs period · auto-page on 2× budget consumption

Incident Response for Agents

DETECT → TRIAGE → MITIGATE · POST-MORTEM



POSTMORTEM OUTPUT

timeline · 5-why · guard gap · eval regen · PR with regression · doc merged to runbook

RUNBOOK DISCIPLINE

every action idempotent · reversible · logged · dry-run before prod · chat-ops archive